



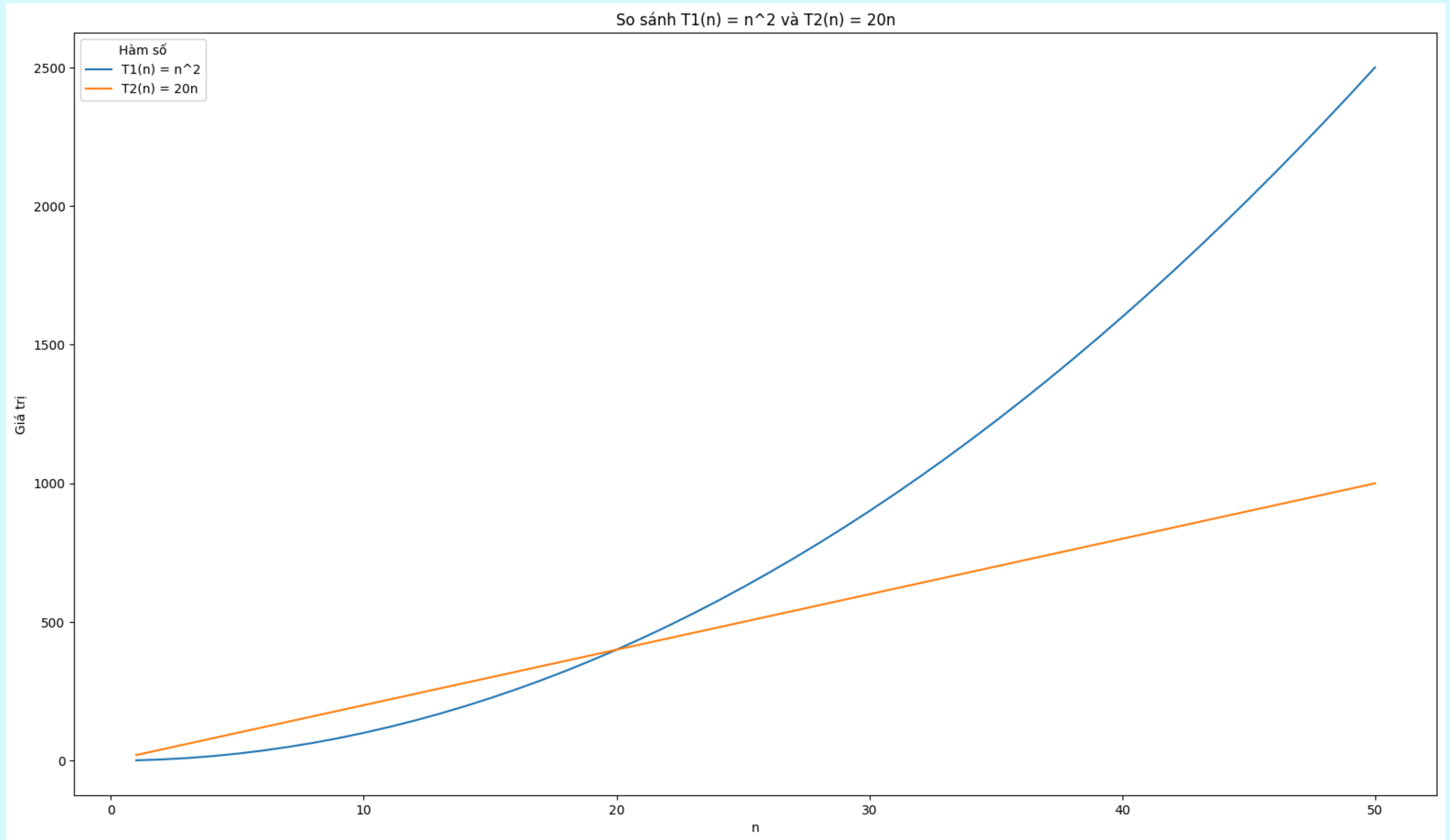
Phân tích độ phức tạp thuật toán không đệ quy

**Căn cứ nào để so sánh tốc độ thực thi giữa
các giải thuật xử lý cùng bài toán ?**

KHÁI NIỆM ĐỘ PHỨC TẠP THUẬT TOÁN

Giả sử thời gian thực hiện của giải thuật đầu tiên là $T1(n) = n^2$,
thời gian thực hiện của giải thuật thứ hai là $T2(n) = 20n$, ta có thể
xem biểu đồ sau :

KHÁI NIỆM ĐỘ PHỨC TẠP THUẬT TOÁN



KHÁI NIỆM ĐỘ PHỨC TẠP THUẬT TOÁN

Độ phức tạp thuật toán (Algorithm Complexity) là thước đo số lượng tài nguyên (thời gian hoặc bộ nhớ) mà một thuật toán cần để giải quyết một bài toán, tính theo kích thước đầu vào N .

Nói cách khác, độ phức tạp cho biết:

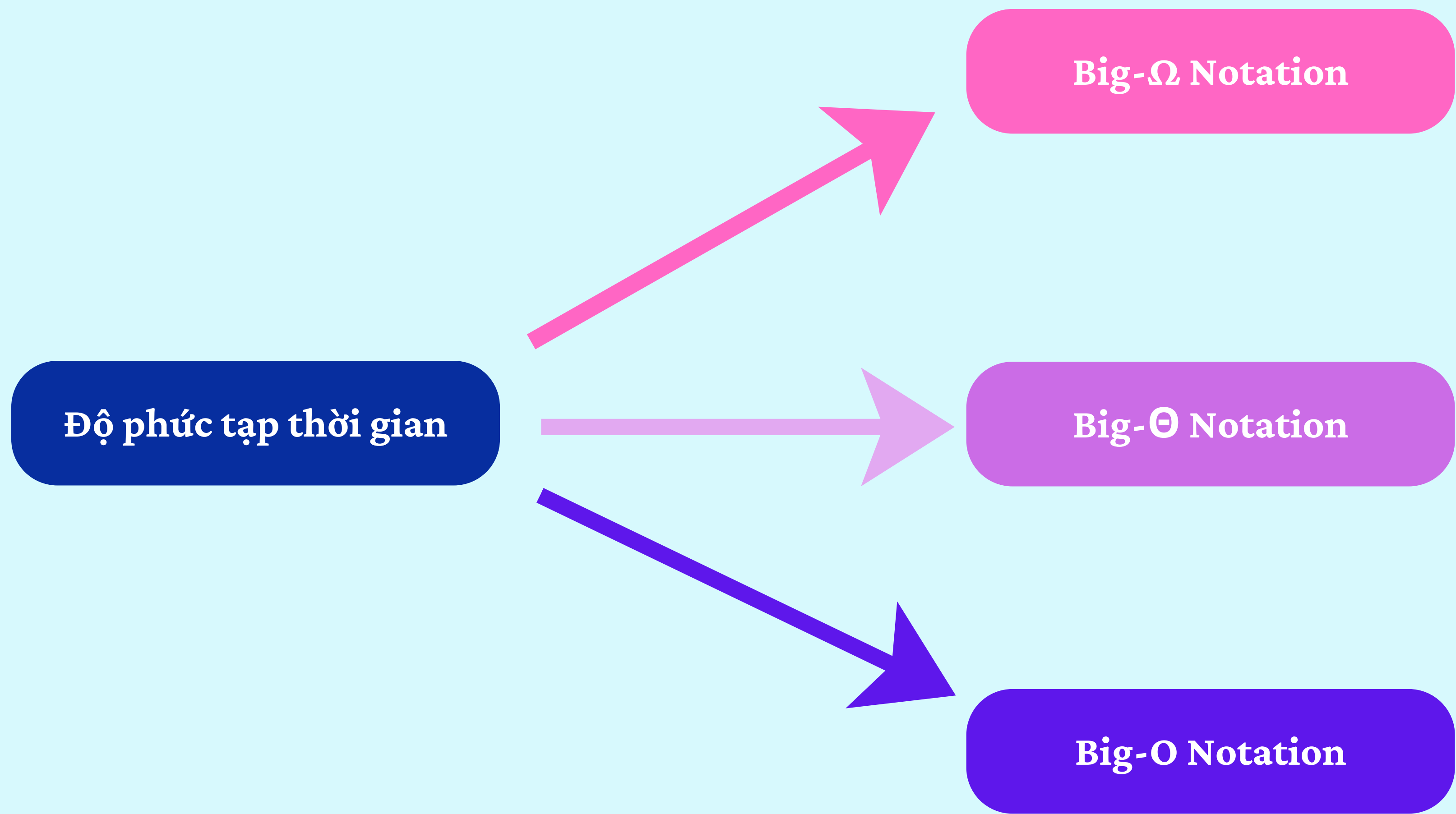
- Thuật toán sẽ chạy nhanh hay chậm (time complexity).
- Thuật toán cần nhiều hay ít bộ nhớ (space complexity).

Độ phức tạp thời gian (Time Complexity)

- Đo lường số phép tính hoặc số bước thực hiện của thuật toán theo kích thước dữ liệu đầu vào n .
- Ví dụ:
 - Thuật toán tìm kiếm tuyến tính: $O(n)$
 - Thuật toán tìm kiếm nhị phân: $O(\log n)$

Độ phức tạp không gian (Space Complexity)

- Đo lường bộ nhớ (RAM) mà thuật toán cần khi chạy.
- Bao gồm: bộ nhớ cho biến, mảng, stack, recursion, ...



BIG-OMEGA NOTATION

Big-Omega Notation

Big-Omega (Ω) Notation là một cách để biểu diễn cận dưới tiệm cận (asymptotic lower bound) của độ phức tạp thời gian của một thuật toán, vì nó phân tích trường hợp tốt nhất (best-case) của thuật toán. Nó cung cấp một giới hạn thấp nhất cho thời gian chạy của một thuật toán theo kích thước đầu vào.

Ký hiệu: $\Omega(f(n))$, trong đó $f(n)$ là một hàm biểu diễn số phép toán (bước xử lý) mà một thuật toán cần để giải quyết một bài toán có kích thước n .

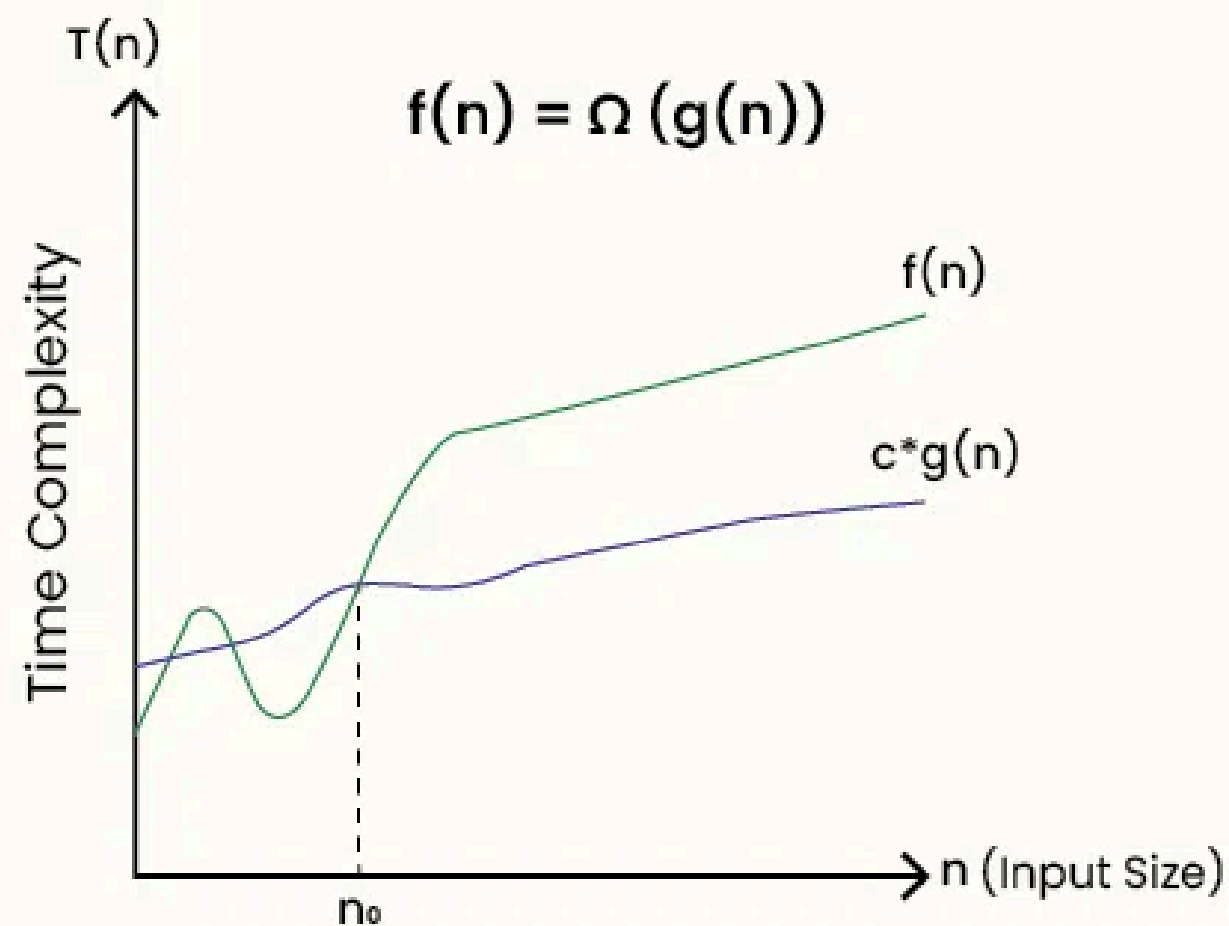
Big-Omega (Ω) Notation được dùng khi ta cần tìm cận dưới tiệm cận của một hàm. Nói cách khác, ta dùng Big-Omega Ω khi muốn biểu diễn rằng thuật toán sẽ tốn ít nhất một lượng thời gian hoặc bộ nhớ nhất định.

Big-Omega Notation

Cho hai hàm $g(n)$ và $f(n)$, ta nói rằng:

$f(n) = \Omega(g(n))$ nếu tồn tại các hằng số $c > 0$ và $n_0 \geq 0$ sao cho:

$$f(n) \geq c \cdot g(n), \forall n \geq n_0$$



Big Omega Ω Notation

$f(n)$ is bounded below
by $g(n)$ asymptotically



BIG-THETA NOTATION

Theta Notation

Trong phân tích thuật toán, các ký pháp tiệm cận (asymptotic notations) được dùng để đánh giá hiệu năng của một thuật toán bằng cách biểu diễn bậc tăng trưởng chính xác (exact order of growth).

Ký hiệu **Big-Theta** (Θ) được biểu diễn bởi chữ cái Hy Lạp Θ .

Theta Notation - Định nghĩa

Cho hai hàm số $f(n)$ và $g(n)$ từ tập số tự nhiên sang số tự nhiên.
Ta nói rằng:

$$f(n) = \Theta(g(n))$$

Nếu tồn tại các hằng số $c_1 > 0$, $c_2 > 0$ và một số tự nhiên n_0 sao cho :

$$c_1 \cdot g(n) \leq f(n) \leq c_2 \cdot g(n), \forall n \geq n_0$$

Theta Notation - Biểu diễn toán học

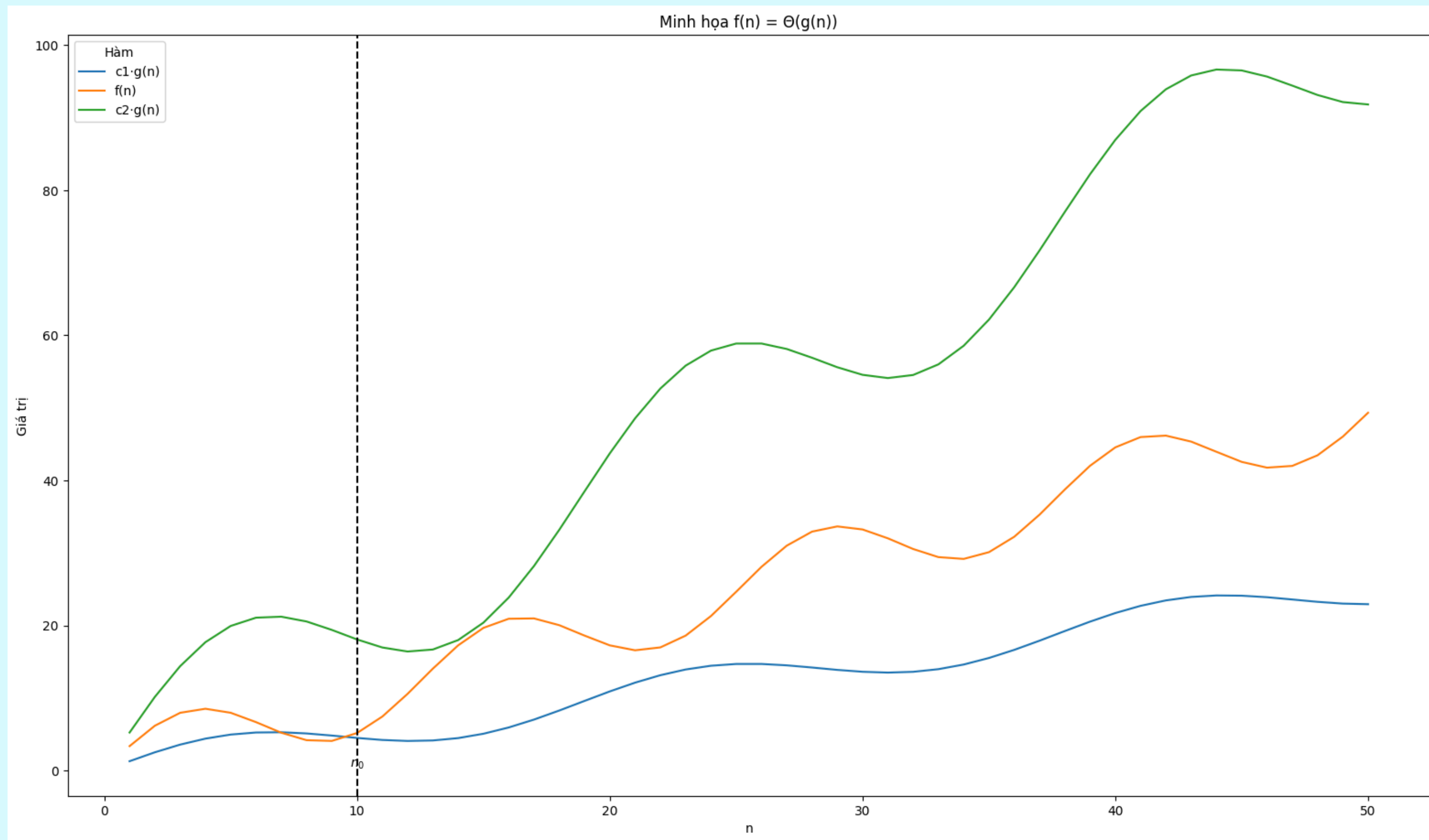
$$\Theta(g(n)) = \{f(n) : \exists c_1, c_2 > 0, n_0 \in \mathbb{N} \text{ sao cho } 0 \leq c_1 g(n) \leq f(n) \leq c_2 g(n), \\ \forall n \geq n_0\}$$

Giải thích đơn giản

Nếu $f(n) = \Theta(g(n))$, điều đó có nghĩa là:

- Với n đủ lớn, giá trị $f(n)$ luôn nằm giữa hai đường biên trên và dưới: $c_1 g(n)$, $c_2 g(n)$
- Hay nói cách khác, $f(n)$ và $g(n)$ tăng trưởng cùng bậc.

Theta Notation



Theta Notation - Ví dụ minh họa

$$1) f(n) = 5n + 3 \quad \longrightarrow \quad f(n) = \Theta(n)$$

$$2) f(n) = 2n^2 + 7n + 4 \quad \longrightarrow \quad f(n) = \Theta(n^2)$$

$$3) f(n) = 6n^6 + 7n^2 + 10 \quad \longrightarrow \quad f(n) = \Theta(n^6)$$

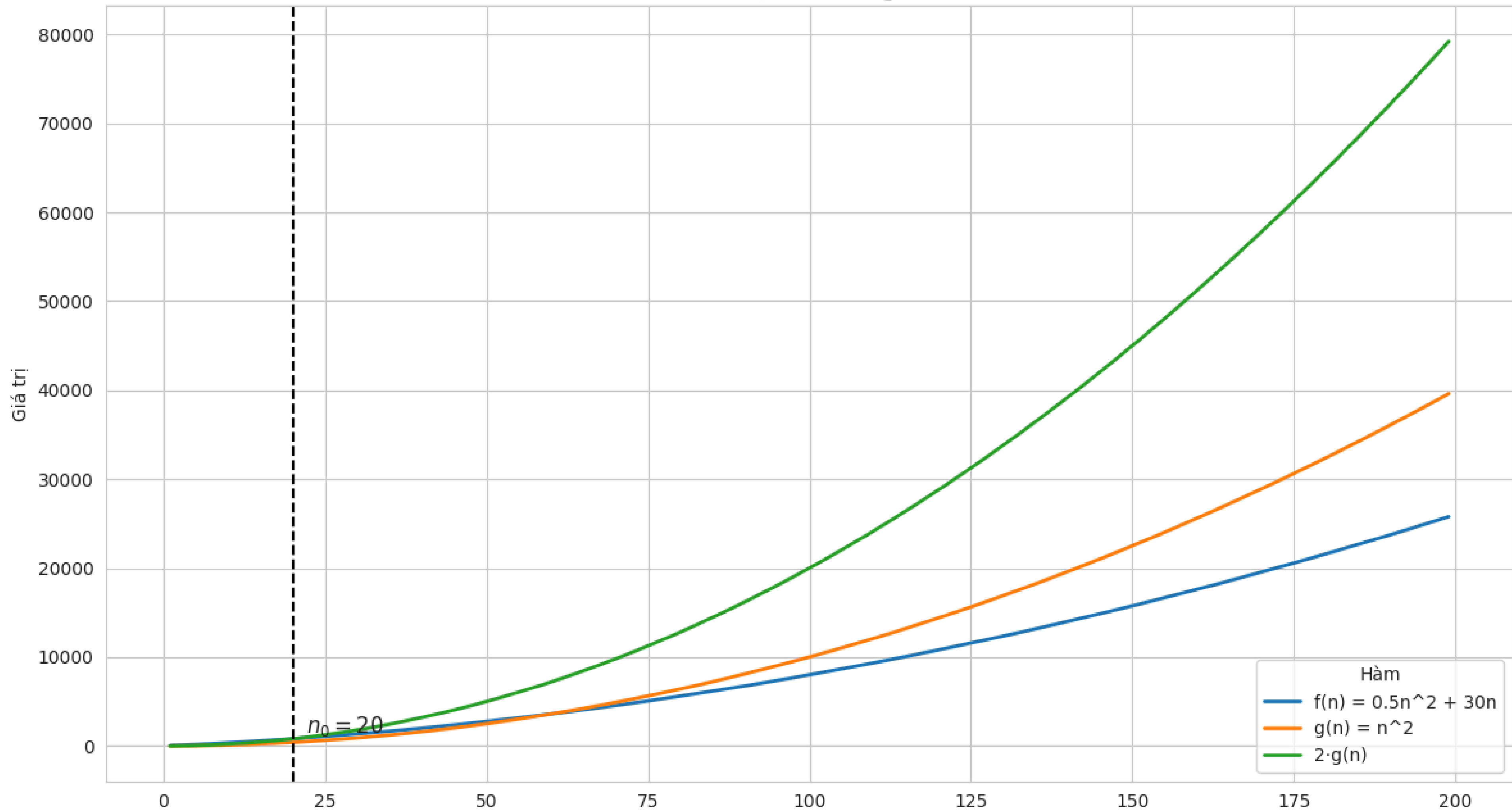
BIG-O NOTATION

Big-O Notation

Cho hai hàm $g(n)$ và $f(n)$, ta nói rằng:
 $f(n) = O(g(n))$ nếu tồn tại các hằng số $c > 0$ và $n_0 \geq 0$ sao cho:

$$f(n) \leq c \cdot g(n), \forall n \geq n_0$$

Minh họa $f(n) = O(g(n))$



Big-O Notation - Cách hiểu đơn giản

- $f(n)$ là $O(g(n))$ nếu tốc độ tăng trưởng của $f(n)$ không vượt quá một bội số hằng số của $g(n)$, khi n đủ lớn.
- Nói cách khác, Big-O cho ta biết cận trên tiệm cận của độ phức tạp.

Big-O Notation - 4 quy tắc tính

1. Quy tắc bỏ hằng số

- Trong phân tích độ phức tạp, các hằng số không ảnh hưởng đến tốc độ tăng trưởng, nên ta bỏ qua.

👉 Ví dụ:

- $f(n) = 5n + 10 \Rightarrow O(n)$
- $f(n) = 1000n^2 \Rightarrow O(n^2)$

2. Quy tắc lấy max

- Khi có nhiều thành phần, bậc cao nhất (tăng nhanh nhất) sẽ quyết định độ phức tạp.

👉 Ví dụ:

- $f(n) = n^2 + n + 1 \Rightarrow O(n^2)$
- $f(n) = n^3 + 1000n^2 + n \Rightarrow O(n^3)$

3. Quy tắc cộng

- Nếu một bài toán gồm nhiều bước chạy nối tiếp, độ phức tạp là tổng của từng bước.
- Sau đó áp dụng quy tắc lấy max để đơn giản hóa.

👉 Ví dụ:

- B1: $O(n^2)$, B2: $O(n \log n)$
- Tổng: $O(n^2 + n \log n) = O(n^2)$

4. Quy tắc nhân

- Giả sử một đoạn chương trình có thời gian thực hiện $T(n) = O(f(n))$.
- Nếu ta thực hiện lặp lại $k(n)$ lần đoạn chương trình đó, trong đó $k(n) = O(g(n))$, thì độ phức tạp tổng thể sẽ là:

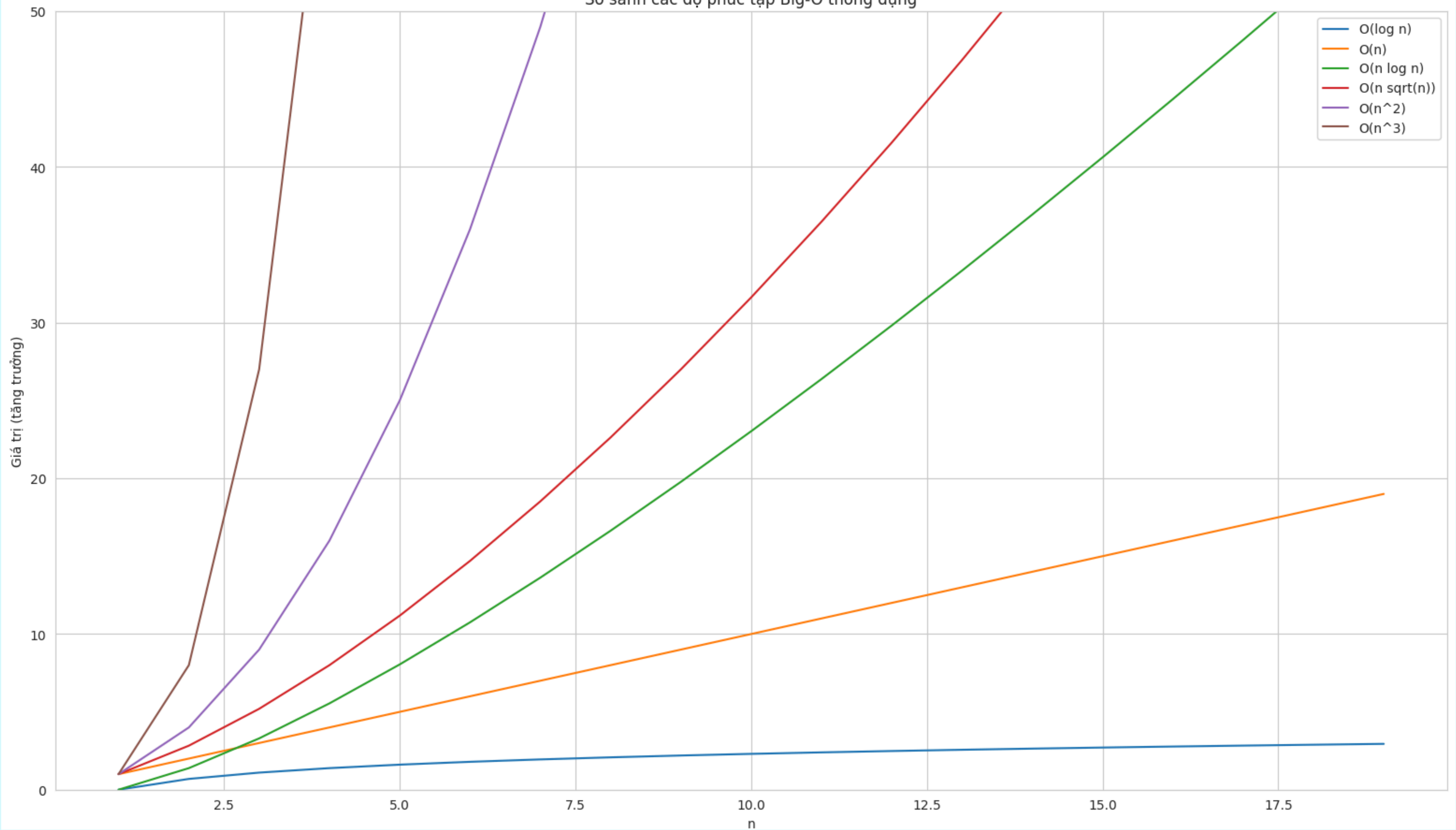
$$T(n) = O(g(n) \cdot f(n))$$

```
for (int i = 0; i < n; i++) {           // O(n)
    for (int j = 0; j < n; j++) {       // O(n)
        // công việc O(1)
    }
}
```

$$O(n) \times O(n) = O(n^2)$$

CÁC HÀM ĐỘ PHỨC TẠP THÔNG DỤNG

So sánh các độ phức tạp Big-O thông dụng



**Tại sao người ta thường sử dụng Big-O
Notation để tính toán độ phức tạp, thay vì
Big-Omega hay Big - Theta Notation ?**

ĐỘ PHỨC TẠP KHÔNG GIAN

SPACE COMPLEXITY

Độ phức tạp không gian

Space Complexity là tổng bộ nhớ mà một thuật toán cần sử dụng khi chạy chương trình, tính theo kích thước đầu vào.

Nó bao gồm hai phần chính: *bộ nhớ để lưu dữ liệu đầu vào* và *bộ nhớ phụ thêm mà thuật toán cần trong quá trình xử lý*.

Ví dụ: nếu ta cần lưu một mảng có kích thước n , thì chỉ riêng phần dữ liệu đã chiếm $O(n)$ bộ nhớ. Nếu ta cần lưu một ma trận, bộ nhớ sẽ là $n \times n$. Đặc biệt, trong các thuật toán đệ quy, mỗi lần gọi hàm sẽ chiếm thêm bộ nhớ stack, do đó độ phức tạp không gian sẽ phụ thuộc vào độ sâu của lời gọi đệ quy.

Nói cách khác, Space Complexity phản ánh mức tiêu tốn bộ nhớ của thuật toán và giúp ta lựa chọn giải pháp vừa nhanh, vừa tiết kiệm tài nguyên.

Nói cách khác, Space Complexity phản ánh mức tiêu tốn bộ nhớ của thuật toán và giúp ta lựa chọn giải pháp vừa nhanh, vừa tiết kiệm tài nguyên.

END.